



MODULE 3

Tools & Action Layer in Agents

Extending LLMs Beyond Text: Function Calling, APIs, and Execution Patterns

Session Roadmap



Role of Tools in Agentic AI

Why LLMs need tools, extending beyond text



Types of Tools

APIs, functions, databases, file systems, services



Tool Invocation Mechanisms

Function calling, JSON schemas, decision strategies



Agent Execution Patterns

Single-step, iterative loops, planner-executor



Frameworks Overview

LangChain, CrewAI, AutoGPT, LangGraph

SECTION 1

Role of Tools in Agentic AI

Why LLMs need external tools to become effective agents

Why LLMs Need External Tools



LLMs are powerful reasoning engines but can only generate text. To access real-time data, perform computation, take actions, and persist state, they need external tools, transforming a text predictor into an autonomous agent.



No Real-Time Data

LLMs have knowledge cutoff.
Tools provide live info.



No Computation

LLMs approximate math.
Tools compute precisely.



No Actions

LLMs generate text only.
Tools execute operations.

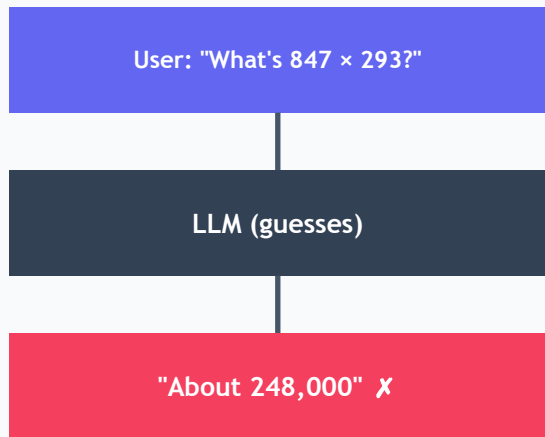


No Persistence

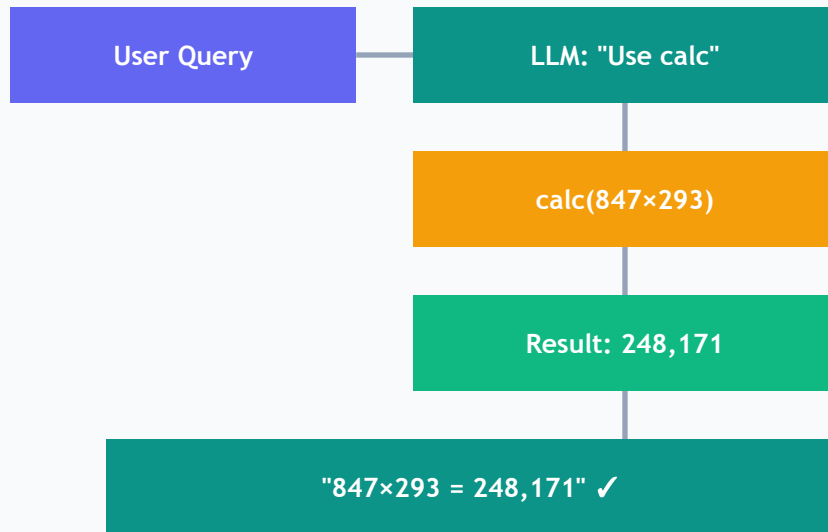
LLMs forget between calls.
Tools read/write data.

LLM Without vs With Tools

Without Tools



With Tools



Tools give LLMs the ability to act on the world with precision and reliability.

Extending Beyond Text Generation

Information Access

Real-time web search, database queries, knowledge base retrieval, API data fetching

Computation

Math calculations, statistical analysis, data transformations, formula evaluation

External Actions

Send emails, update CRM, create tickets, trigger workflows, deploy code

File Operations

Read documents, generate reports, create spreadsheets, process images, parse PDFs

Code Execution

Run Python scripts, execute SQL, test code, process data pipelines in sandboxed environments

SECTION 2

Types of Tools

APIs, functions, databases, file systems, and external services

Tool Type: APIs



REST and GraphQL APIs allow agents to interact with external services, fetching data, triggering actions, and integrating with the ecosystem.

Data APIs

OpenWeatherMap, Alpha Vantage, NewsAPI, Google Maps

Real-time info retrieval

Action APIs

Twilio (SMS), SendGrid (email), Stripe (payment processing), GitHub

Execute external operations

AI/ML APIs

Google Vision, HuggingFace Inference, ElevenLabs

Specialized AI tasks

SaaS APIs

Salesforce, HubSpot, Notion, Jira, Slack

Business workflow integration

Search APIs

Google, Bing, Serper, Tavily, Brave Search

Web-scale info retrieval

Tool Type: Functions & Calculators



Local functions provide precise computation, data transformation, and logic, things LLMs struggle with.

Function Execution Flow:



Math Operations

Arithmetic, algebra, statistics, exact results, not approximations

Logic Processors

Boolean expressions, constraints, validation, decision trees

Data Transforms

Parse dates, convert units, format currencies, sort/filter, regex

String Tools

Text formatting, URL encoding, hash generation, templates

Tool Type: Databases & File Systems



Persistent storage tools let agents read, write, query, and manage data across sessions.

SQL Databases

Execute queries against PostgreSQL, MySQL, SQLite. Structured data access with full SQL support.

Vector Databases

Semantic search over embeddings, Pinecone, Weaviate, ChromaDB. Powers RAG and long-term memory.

NoSQL / Document

MongoDB, Firestore, DynamoDB, flexible document storage for unstructured agent data.

File System I/O

Read/write files, list directories, create documents, parse CSVs, process PDFs and images.

Object Storage

S3, GCS, Azure Blob — store and retrieve large files, images, datasets at scale.

SECTION 3

Tool Invocation Mechanisms

How agents discover, select, and call tools

Function Calling: Core Mechanism



Function calling is the native API capability allowing LLMs to output structured JSON tool invocations. The model selects the function and generates arguments, the orchestration layer executes it.

1. Register Tools

2. Send to LLM

3. LLM Selects Tool + Args

4. Execute Function

5. Return Result

Model-Native

Built into GPT-4, Claude, Gemini — structured function calling

Multi-Tool

Models select from dozens of tools per request automatically

Type-Safe

JSON Schema defines types, required fields, validation rules

Parallel Calls

Some models call multiple tools simultaneously

JSON Schema Tool Definition

Tools are defined with JSON Schema — the model sees these and learns how to call each tool.

Tool Definition:

```
{
  "name": "get_weather",
  "description": "Get current
    weather for a city",
  "parameters": {
    "type": "object",
    "properties": {
      "city": {
        "type": "string",
        "description": "City name"
      },
      "units": {
        "type": "string",
        "enum": ["celsius",
          "fahrenheit"]
      }
    },
    "required": ["city"]
  }
}
```

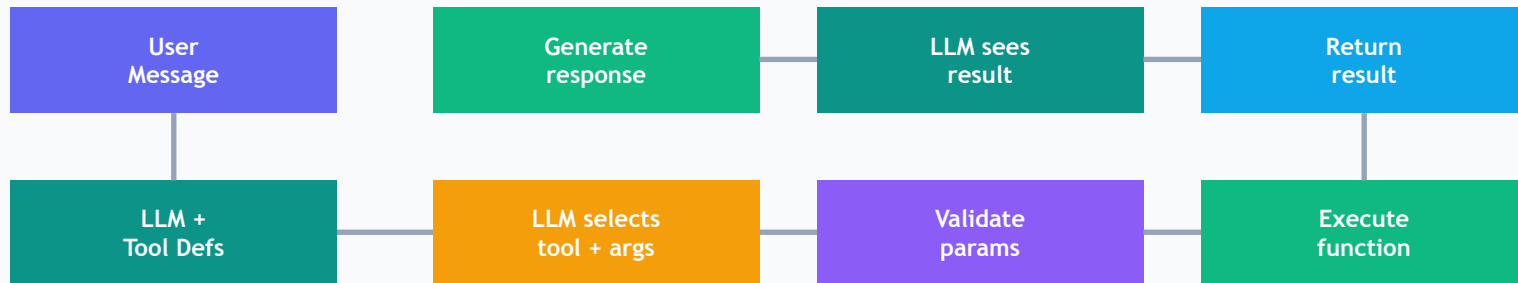
LLM Output:

```
{
  "tool": "get_weather",
  "arguments": {
    "city": "Tokyo",
    "units": "celsius"
  }
}
```

Key Schema Elements:

- name — unique identifier
- description — when to use
- parameters — JSON Schema
- required — mandatory fields
- enum — constrained choices

Tool Call Lifecycle



Error Handling in the Lifecycle:

- **Validation Failure:** Invalid params → error to LLM → regenerates with fixes
- **Execution Error:** API timeout/500 → error msg → LLM retries or tries alternative
- **No Tool Match:** No suitable tool → respond with text or ask for clarification
- **Rate Limiting:** Too many calls → queue, backoff, notify user of delay

Prompt-Based Tool Selection

Before native function calling, tools were invoked through prompt engineering, the model outputs tool calls in text format.

ReAct-Style Prompt Approach

System: Tools available:

- search(query) – web search
- calculate(expr) – evaluate math

User: What is France GDP times 2?

Thought: I need France GDP, then multiply.

Action: search("GDP of France 2024")

Observation: \$3.13 trillion

Thought: Now calculate 3.13×2

Action: calculate("3.13 * 2")

Observation: 6.26

Answer: ~\$6.26 trillion.

Decision: When to Use Tools



The agent must decide: use a tool or respond directly? This is encoded in the system prompt.

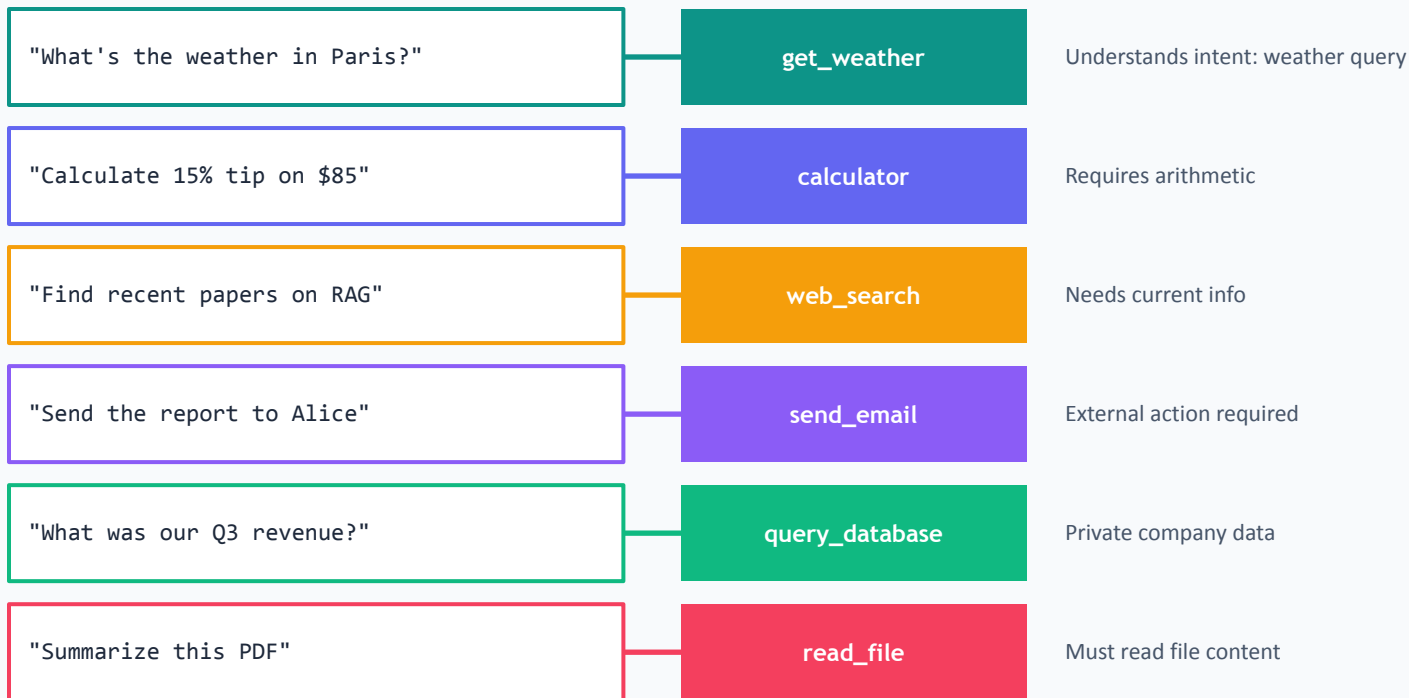


Use a Tool When:

Current information needed • Precise computation required • External action must be taken • Data must be read/written • Answer depends on private data not in training

Decision: Which Tool to Use

The LLM uses tool descriptions and context to select the best tool. Good descriptions are critical.



Writing Effective Tool Descriptions

Bad Description

✗ "Gets weather data"

✓ "Get current weather for a city including temperature, humidity, wind. Use when user asks about weather or temperature."

Bad Param Names

✗ "data", "input", "value"

✓ "city_name", "date_range", "currency_code" – self-documenting names

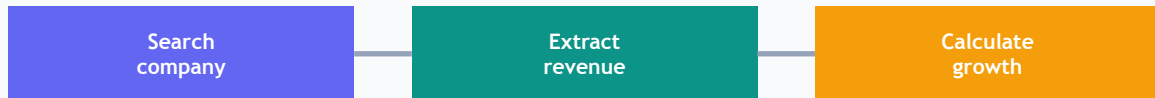
Missing Constraints

✗ No enum, no format hints

✓ "units": { "enum": ["celsius", "fahrenheit"] } – constrain to valid options

Parallel vs Sequential Calls

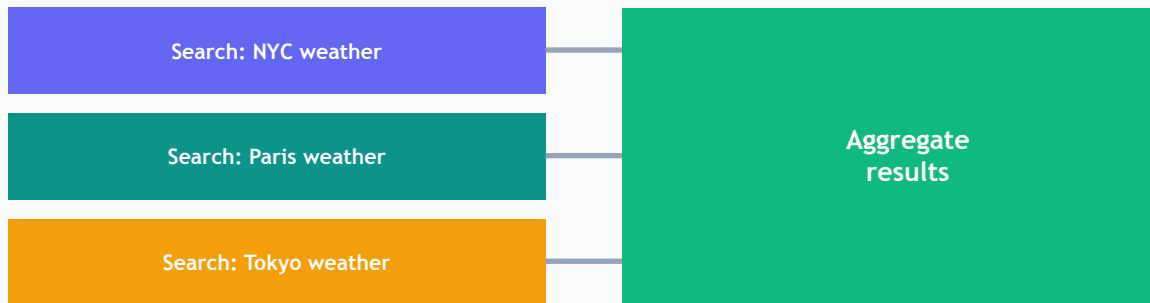
Sequential (dependent):



Each step depends on previous → sequential

Parallel (independent):

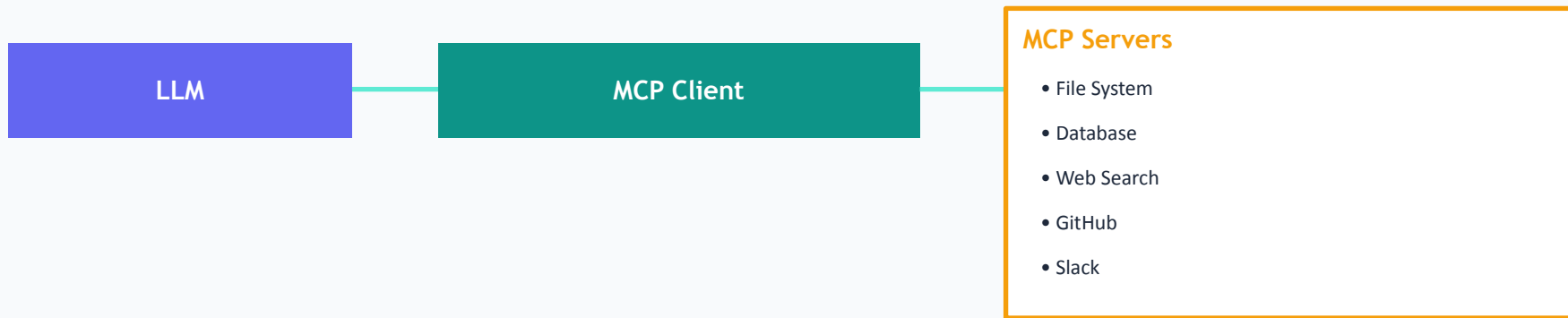
All 3
at once



MCP: Model Context Protocol



MCP is an open standard by Anthropic — a universal protocol for connecting LLMs to tools. Think "USB-C for AI" — one interface for any model and any tool.



Standardized

One protocol across all LLM providers

Composable

Mix and match MCP servers freely

Secure

Local/remote with explicit permissions

MCP: Model Context Protocol

```
# weather_server.py
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("weather-tools")

# Fake API call (simulates external service)
def call_weather_api(city: str, units: str = "celsius") -> dict:
    # Simulated API response
    return {
        "city": city,
        "temperature": 22,
        "units": units,
        "condition": "sunny"
    }

@mcp.tool()
def get_weather(city: str, units: str = "celsius") -> str:
    """Get current weather for a city using external API"""

    response = call_weather_api(city, units)

    return f'{response["city"]}: {response["temperature"]}° {response["units"]}, {response["condition"]}'

if __name__ == "__main__":
    mcp.run()
```

LLM Generates:

```
{
  "tool": "get_weather",
  "arguments": {
    "city": "Tokyo",
    "units": "celsius"
  }
}
```

MCP Executes and Returns:

```
{
  "city": "Tokyo",
  "temperature": 22,
  "units": "celsius",
  "condition": "sunny"
}
```

SECTION 4

Agent Execution Patterns

How agents orchestrate tool calls into workflows

Pattern 1: Single-Step Execution

Simplest pattern — one LLM call, zero or one tool call, direct response.



Pros

- Simple to implement
- Low latency
- Predictable
- Minimal cost

Cons

- Can't handle multi-step
- No error recovery
- Limited complexity
- No context accumulation

Pattern 2: Iterative Reasoning Loop

Loop: Observe → Think → Act → Evaluate until goal met or stopping condition hit.



Example: "Research AWS vs GCP pricing for ML workloads"

Iter 1	Search AWS ML pricing
Iter 2	Search GCP ML pricing
Iter 3	Calculate cost differences
Iter 4	Generate comparison table → DONE

Pattern 3: Planner-Executor

Separate planning from execution; planner creates full plan, executor carries it out step by step.



Example Plan: "Analyze top 5 competitors"

Step 1: Search for top 5 competitors → `web_search`

Step 2: Gather revenue/market share per competitor → `web_search × 5`

Step 3: Query our internal metrics → `query_database`

Step 4: Calculate comparative metrics → `calculator`

Step 5: Generate comparison report → `create_document`

ReAct Pattern for Tool Use

Interleaves explicit reasoning with tool actions, creating a traceable execution log.

Thought

User needs weather forecast for Barcelona trip.

Action

```
get_weather(city="Barcelona", forecast_days=3)
```

Result

Mon: 28°C sunny | Tue: 25°C cloudy | Wed: 22°C rain

Thought

Wednesday has rain – suggest umbrella + indoor activities.

Action

```
search("indoor activities Barcelona")
```

Result

Sagrada Familia, Picasso Museum, La Boqueria...

Thought

Have weather + indoor alternatives for rainy day. Ready to respond.

Answer

Here's your 3-day Barcelona forecast with activity recommendations...

Tool Safety & Guardrails



Permission Scoping

Define exactly which tools each agent can access — research agent shouldn't have payment API

Input Validation

Validate all parameters before execution — check types, ranges, sanitize for injection

Confirmation Gates

For destructive actions (delete, send, deploy), require explicit user confirmation

Rate Limiting

Limit tool calls per minute/session to prevent runaway loops or accidental DDoS

Audit Logging

Log every tool call with input, output, timestamp — essential for debugging and compliance

Sandboxing

Execute agent-generated code in isolated containers — never on production systems

Error Handling & Retry Strategies

Retry Same Tool

Network timeout → wait 2s → retry with exponential backoff: 1s, 2s, 4s, 8s. Max 3 retries.

Retry Different Args

"Not found" → rephrase query, try alternative parameters, broaden the search scope.

Fallback to Different Tool

Primary tool fails → try alternative. `web_search` → `web_fetch` → cached data.

Ask for Clarification

Ambiguous input → ask user. "Did you mean Paris, France or Paris, Texas?"

Graceful Degradation

All tools fail → provide best answer from LLM knowledge + explain live data unavailable.

Tool Orchestration Patterns

Sequential Chain

$A \rightarrow B \rightarrow C$ — each output feeds next. For dependent steps.

Parallel Fan-Out

Call N tools simultaneously when independent, then aggregate.

Conditional Branch

Choose tools based on intermediate results — if X, use Y; else Z.

Recursive/Iterative

Same tool called N times with different inputs — pagination, batching.

Map-Reduce

Fan out same tool across N inputs, then reduce/aggregate all results.

SECTION 5

Frameworks Overview

LangChain, CrewAI, AutoGPT, and the agent ecosystem

LangChain



The most popular framework for LLM apps: modular components for chains, tools, memory, and output parsers.

Chains

Sequential pipelines: prompt → LLM → parser → next step

Agents

Dynamic tool selection based on task and context

Tools

Pre-built: search, calc, SQL, file I/O + custom tools

Memory

Buffer, summary, entity, vector store memory

Retrievers

Document loaders + splitters + vector stores = RAG

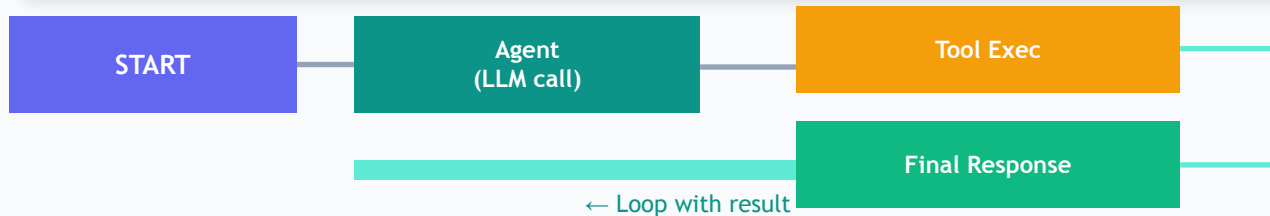
Output Parsers

JSON, Pydantic models, lists, custom formats

LangGraph: Stateful Workflows



Extends LangChain with graph-based execution — nodes, edges, cycles, conditional branching, persistent state.



Cycles & Loops

Graphs support loops: essential for iterative agents

Persistent State

Checkpoint, serialize, resume: pause and continue later

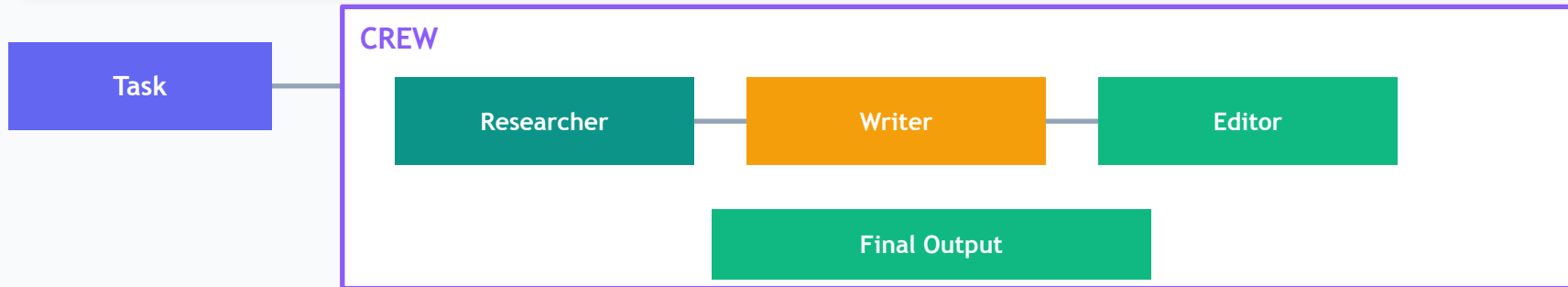
Human-in-Loop

Approval nodes where execution pauses for human review

CrewAI: Role-Based Agent Teams



Organizes agents into crews with roles, goals, and tools. Each agent has a specialty — they collaborate as a team.



Role Assignment

Each agent has title, goal, and backstory shaping behavior

Task Delegation

Tasks flow between agents sequentially or by capability

Building Custom Tools

Most real-world agents need custom tools tailored to your domain.

1

Define Interface

Name, description, parameters (JSON Schema), return type. Clear descriptions are critical.

2

Implement Logic

Write function body — API calls, queries, calculations. Handle errors gracefully.

3

Validate Inputs

Check types, ranges, allowlists. Never trust LLM-generated inputs blindly.

4

Format Output

Return structured results the LLM can parse. Include relevant data, exclude noise.

5

Test Extensively

Diverse inputs, edge cases, error conditions. Verify LLM selects tool correctly.

Tools in the Agent Stack — Recap

Input: User queries, system events

Reasoning: LLM decides what to do

TOOLS: Execute actions on the real world (THIS MODULE)

Memory: Context + persistence

Output: Deliver results to user

Tools bridge reasoning and action — transforming text generators into capable agents.

Key Takeaways

- 1 LLMs need tools for real-time data, computation, actions, and persistence
- 2 Tool types: functions, APIs, databases, file I/O, browser, MCP servers
- 3 Function calling provides structured, type-safe tool invocation via JSON Schema
- 4 Tool descriptions are the #1 factor in selection accuracy — invest in them
- 5 Three patterns: single-step, iterative loop, planner-executor
- 6 Safety: validation, confirmation gates, rate limits, sandboxing, logging
- 7 LangChain, LangGraph, CrewAI, MCP accelerate development



Thank You!!!

